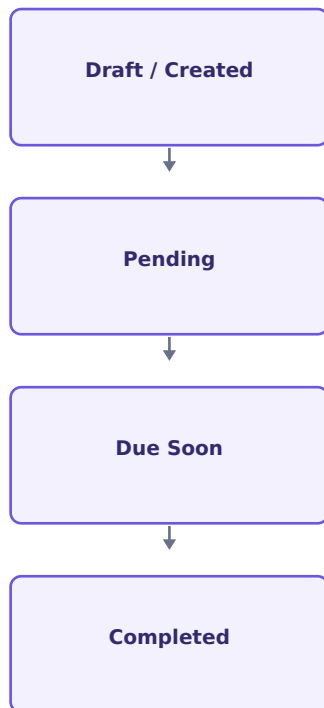


The production design avoids depending on a local `client_secret.json` file on Render by supplying OAuth client configuration through environment variables and persisting the temporary OAuth transaction state server-side.

9. UML-Style State Models

Task lifecycle



Calendar connection state

State	Meaning
Not connected	No active Google Calendar connection for the user.
Connecting	OAuth transaction is in progress; state/verifier are temporary.
Connected	Authorized credentials are associated with the user's account.
Disconnected	Connection has been removed/disabled and UI returns to Not connected.

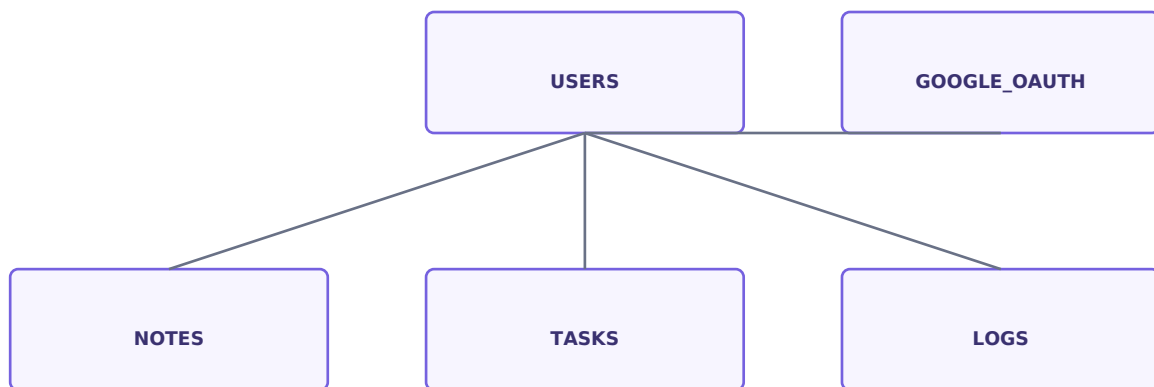
Note interaction state

State	Action	Result
Grid	Eye/View	Open read-only view modal
View modal	Close	Return to grid
View modal	Edit Note	Close view + open editor for same note
Editor	Save	Persist and refresh grid
Grid	Delete	Open confirmation modal

10. MongoDB Data Model / ER-Style View

users Identity, profile, verification state, preferences and Google Calendar connection state.	notes user_id, title, content, category, pinned, created_at, updated_at.
tasks user_id, title, description, category, priority, due date/time, completion and optional calendar event id.	logs user_id plus action/entity information for activity history.
google_oauth Temporary OAuth state and PKCE verifier records used during Calendar connection.	

Logical relationships



The key multi-user rule is data isolation: user-facing queries should include the authenticated user's identifier. This prevents accidental cross-user reads and writes.

11. API / Route Responsibilities

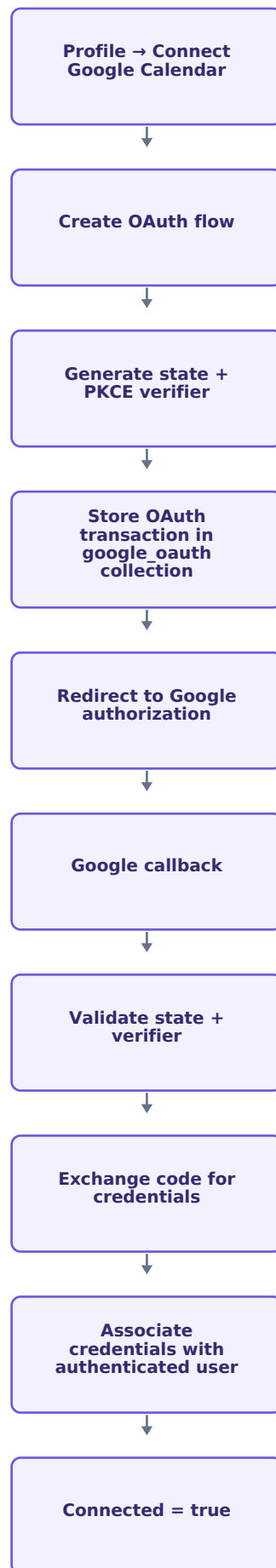
Endpoint	Method	Purpose
/api/auth/me	GET	Authenticated user information
/api/auth/login	POST	Authenticate user
/api/auth/logout	POST	End session
/api/auth/change-password	POST	Change password
/api/profile	GET / PUT	Read/update profile
/api/profile/photo	POST / DELETE	Upload/remove photo
/api/tasks	GET / POST	Read/create tasks
/api/notes	GET / POST	Read/create notes
/api/dashboard/stats	GET	Dashboard/profile statistics
/api/google/status	GET	Calendar connection status
/api/google/connect	GET	Start Google OAuth
/api/google/callback	GET	Complete Google OAuth
/api/google/disconnect	POST	Disconnect Calendar

API pattern

```
Browser
-> shared API helper
-> Flask route
-> login_required
-> validation / business logic
-> MongoDB or Google API
-> JSON response
-> page-specific JavaScript updates UI
```

12. Google Calendar Integration

OAuth architecture



Production callback

The deployed callback path used by the project is:

```
https://mynotestasks.onrender.com/api/google/callback
```

Google OAuth requires the deployed redirect URI to match the configured OAuth client exactly. A mismatch produces Google's `redirect_uri_mismatch` error.

Why no `client_secret.json` is required on Render

The production design can supply the Google OAuth client configuration through environment variables rather than baking a local credentials file into the deployed source. Temporary state and PKCE verifier information are persisted server-side in MongoDB.

13. Calendar Event & Reminder Logic

Exact due-time principle

The task's due time remains the event start time. Reminder offsets are relative alarms and do not replace the due time.

Timeline	Meaning
5:40 PM	10-minute reminder for a 5:50 PM task
5:45 PM	5-minute reminder for a 5:50 PM task
5:50 PM	Exact event/task due time

Event structure

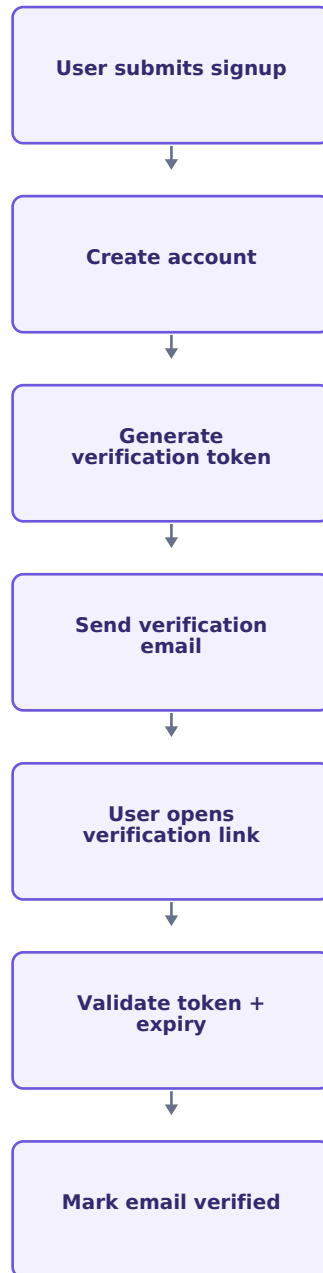
```
event = {
  "summary": title,
  "description": description,
  "start": {
    "dateTime": start_dt.isoformat(),
    "timeZone": timezone_name
  },
  "end": {
    "dateTime": end_dt.isoformat(),
    "timeZone": timezone_name
  },
  "reminders": {
    "useDefault": False,
    "overrides": [
      {"method": "popup", "minutes": 10},
      {"method": "popup", "minutes": 5}
    ]
  }
}
```

Google Calendar supports event-specific reminder overrides, with minutes representing the number of minutes before the event starts. The current project uses popup reminders at 10 and 5 minutes. Actual notification delivery can still depend on the user's Calendar/device notification behavior.

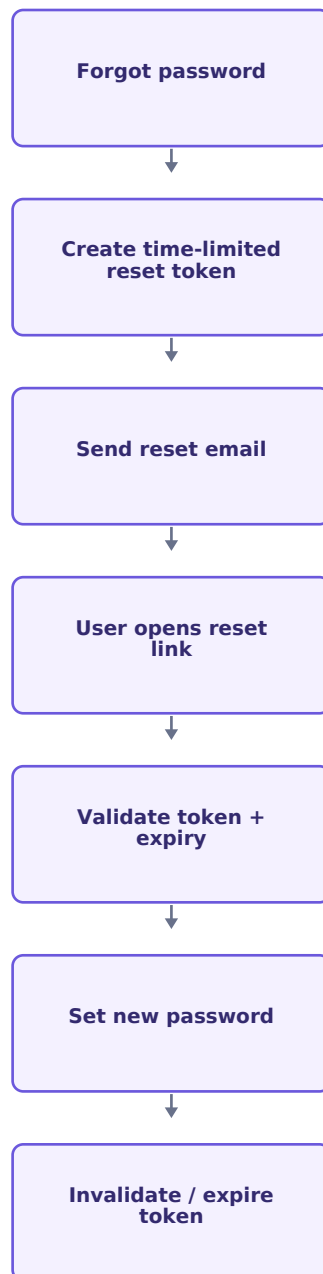
Reference: [Google Calendar API reminders and event resources](#).

14. Email & Account Recovery Workflows

Email verification



Password reset



Task email reminder

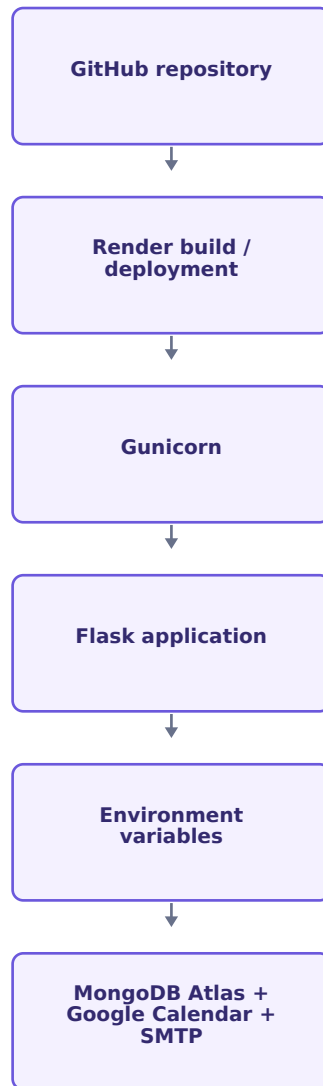
The reminder worker checks eligible tasks, sends the email to the authenticated user's configured recipient, and marks the reminder as sent after successful delivery. The project worker loop checks on a short interval so due reminders can be detected without requiring the browser to stay open.

15. Security & Production Checklist

Secrets Keep MongoDB URI, SMTP password, OAuth client secret/config and Flask secret key in environment variables.	Password storage Use secure password hashing; never store plaintext passwords.
Sessions Use secure cookies, HTTPS, appropriate session lifetime and a protected application secret.	OAuth Validate state and PKCE verifier during callback; expire/delete temporary OAuth transactions.
Authorization Every protected route should require the authenticated user.	Data isolation Filter notes/tasks/profile/log queries by authenticated user identity.
Input validation Validate titles, content, categories, dates and times on the server.	Deployment logs Trace Render 500 errors to the first Python exception.

Important: Never publish a real MongoDB connection string, SMTP password, OAuth secret or Flask SECRET_KEY in GitHub.

16. Deployment Architecture



Production configuration

Variable / setting	Purpose
SECRET_KEY	Flask session/application secret
MONGO_URI	MongoDB Atlas connection
MAIL_HOST / MAIL_PORT	SMTP server configuration
MAIL_USERNAME / MAIL_PASSWORD	SMTP authentication
APP_BASE_URL	Public application base URL
VERIFICATION_EXPIRY_MINUTES	Verification token lifetime
RESET_EXPIRY_MINUTES	Password reset token lifetime
PORT	Render/application listening port
Google OAuth configuration	Client id/secret/project configuration supplied through deployment environment

Deployment validation

- Confirm Render environment variables are present and valid.
- Confirm Google OAuth authorized redirect URI exactly matches the deployed callback.
- Confirm MongoDB Atlas network access permits the deployed application.
- Check Gunicorn startup logs.
- Test login, notes, tasks, Calendar connection and reminders after deployment.

17. Testing & Troubleshooting Guide

Area	Test	Expected result
Login	POST /api/auth/login	200 response and dashboard session
Profile	GET /api/auth/me	Authenticated user returned
Notes	Create/view/edit/pin/delete	UI updates without full page refresh
Notes	Search/filter/sort	Visible cards change correctly
Tasks	Create due date/time	Task appears in dashboard/upcoming
Calendar	Connect Google	Profile status becomes Connected
Calendar	Create event	Event appears in primary calendar
Reminders	5:50 PM test event	10 and 5 minute reminder overrides exist
Disconnect	POST /api/google/disconnect	Connection removed; status Not connected
Deployment	Inspect Render logs	No uncaught 500 exceptions

Important debugging lessons captured during development

- A missing ``json`` import caused OAuth configuration parsing to fail.
- An undefined ``google_oauth_collection`` caused the OAuth connect route to fail.
- Browser-session-only PKCE state was unreliable in the deployed flow; server-side OAuth transaction persistence was adopted.
- The profile JavaScript referenced an undefined ``apiRequest``; the existing API helper should be used consistently.
- Google `redirect_uri_mismatch` means the configured callback URI and deployed callback must match exactly.

18. Recommended Project Structure

```
MyNotesTasks/
├── app.py
├── requirements.txt
├── README.md
├── LICENSE
├── .gitignore
├── .env.example
├── templates/
│   ├── index.html
│   ├── login.html
│   ├── signup.html
│   ├── dashboard.html
│   ├── notes.html
│   ├── tasks.html
│   ├── upcoming.html
│   ├── logs.html
│   ├── profile.html
├── static/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   ├── api.js
│   │   ├── dashboard.js
│   │   ├── notes.js
│   │   ├── tasks.js
│   │   ├── upcoming.js
│   │   ├── logs.js
│   │   └── profile.js
├── docs/
└── project-documentation.pdf
```

Repository responsibilities

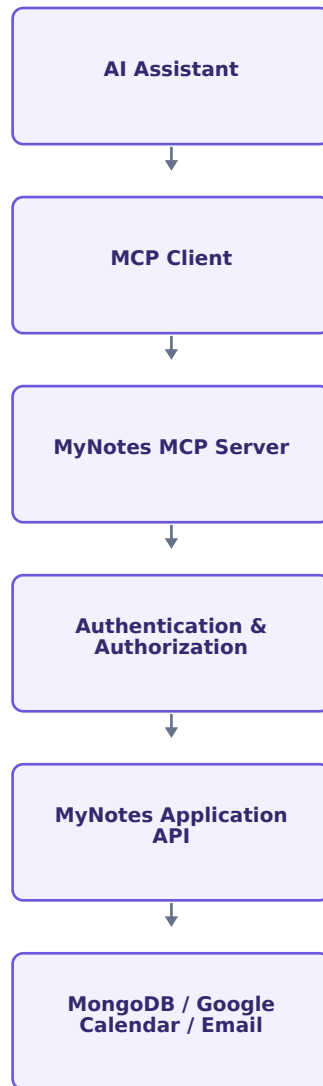
Path	Responsibility
app.py	Flask routes, authentication, business logic, persistence and integrations
templates/	Jinja-rendered application pages
static/js/	Page-specific browser behavior and API calls
static/css/	Shared visual system and responsive UI
requirements.txt	Pinned Python dependencies
.env.example	Safe configuration template without real secrets
docs/	Long-form technical documentation and future architecture

19. Future: MCP Integration

MCP (Model Context Protocol) integration is planned for a future version of MyNotes & Tasks. The future MCP layer is intended to allow compatible AI assistants to interact with the user's productivity workspace through controlled, structured tools.

- AI-powered Notes access
- AI-powered Task management
- Google Calendar interaction
- Smart reminder management
- Unified Notes & Tasks search
- Productivity statistics
- Natural-language task creation
- AI task planning and breakdown
- AI-assisted calendar scheduling
- Task and Calendar synchronization
- Activity and productivity summaries
- Permission-based MCP access
- User-specific data isolation
- MCP Tools, Resources and Prompts
- Future MCP event/notification support

Planned architecture



Roadmap status: MCP is future work and is not presented as an implemented production feature.

20. Engineering Quality, Limitations & Next Steps

Current strengths

- Clear separation between browser UI, Flask APIs and persistence.
- User-specific MongoDB filtering is part of the design.
- Google OAuth is adapted for deployment without requiring a local credentials file on Render.
- Calendar event reminders are configured as relative offsets while preserving the exact task due time.
- Notes view and edit states are intentionally separated.
- Page-specific JavaScript keeps modules focused.

Recommended hardening

- Encrypt or otherwise protect stored long-lived OAuth credential data at rest.
- Add explicit CSRF protection where applicable to state-changing browser requests.
- Add automated unit/API tests and CI checks.
- Add indexes for user_id + due date / updated time as data grows.
- Add rate limiting for authentication and password reset endpoints.
- Add structured production logging and error monitoring.
- Add automated cleanup/TTL policy for temporary OAuth transaction records.

Future product extensions

- Recurring tasks and multiple calendars.
- Custom reminder intervals and browser/push notifications.
- Advanced task filters and labels.
- Rich text / Markdown notes and attachments.
- Calendar view and analytics dashboard.
- Backup/import/export and PWA/mobile support.
- MCP and AI productivity assistant integration.

21. Project Links, License & Documentation

Resource	Link
Live Application	https://mynotestasks.onrender.com/
GitHub Repository	https://github.com/Aluval/MyNotesTasks
Developer GitHub	https://github.com/Aluval
Portfolio	https://harsha24.online/
Developer Search	https://www.google.com/search?q=ALUVALA+EDIGA+HARSHA+VARDHAN+GOUD

Recommended open-source license

Apache License 2.0 is recommended for this public GitHub project because it permits use, modification and redistribution while also providing an explicit patent grant.

README positioning

The public README should describe implemented features as implemented and keep MCP under a clearly labeled future roadmap section. This avoids claiming functionality that is not yet present.

22. Conclusion

MyNotes & Tasks demonstrates an end-to-end productivity application built around practical web engineering: authenticated user accounts, persistent notes and tasks, browser-side state management, email workflows, Google Calendar OAuth, calendar event creation, reminders, activity tracking and cloud deployment.

The application is structured so that future AI/MCP capabilities can be added without changing its core purpose. The most important production principles are user-specific authorization, secure secret management, correct OAuth state handling, exact timezone-aware scheduling and reliable deployment configuration.

This document is intended to serve as a software project report, technical reference, architecture record and GitHub project documentation companion.

Developer: Aluvala Ediga Harsha Vardhan Goud

GitHub: github.com/Aluval/MyNotesTasks

Appendix A — Project Screenshots

The following screenshots were supplied during the project's development. They are placed at the end of the document as requested.

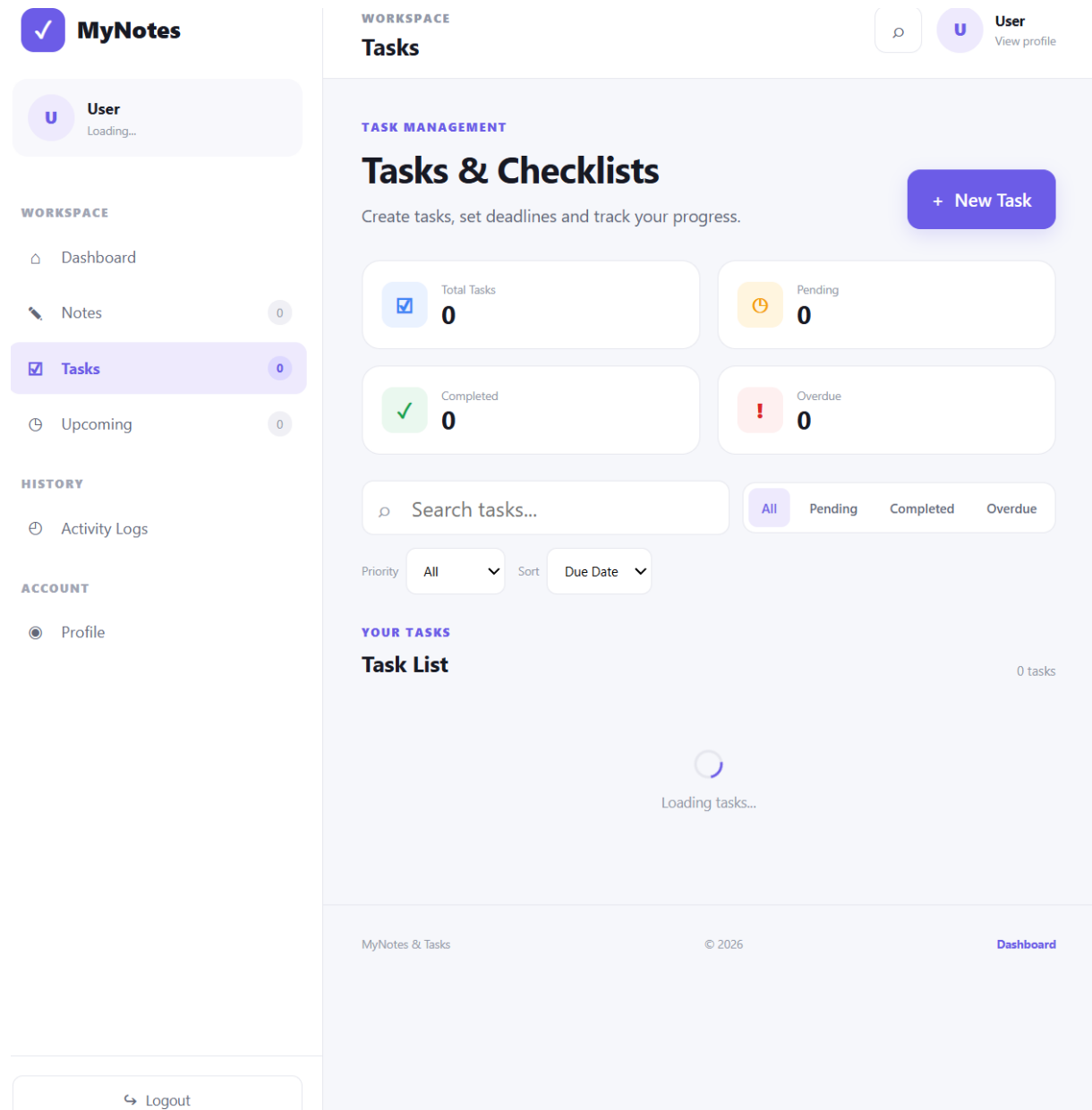


Figure A1. Tasks & Checklists page

Appendix A — Project Screenshots

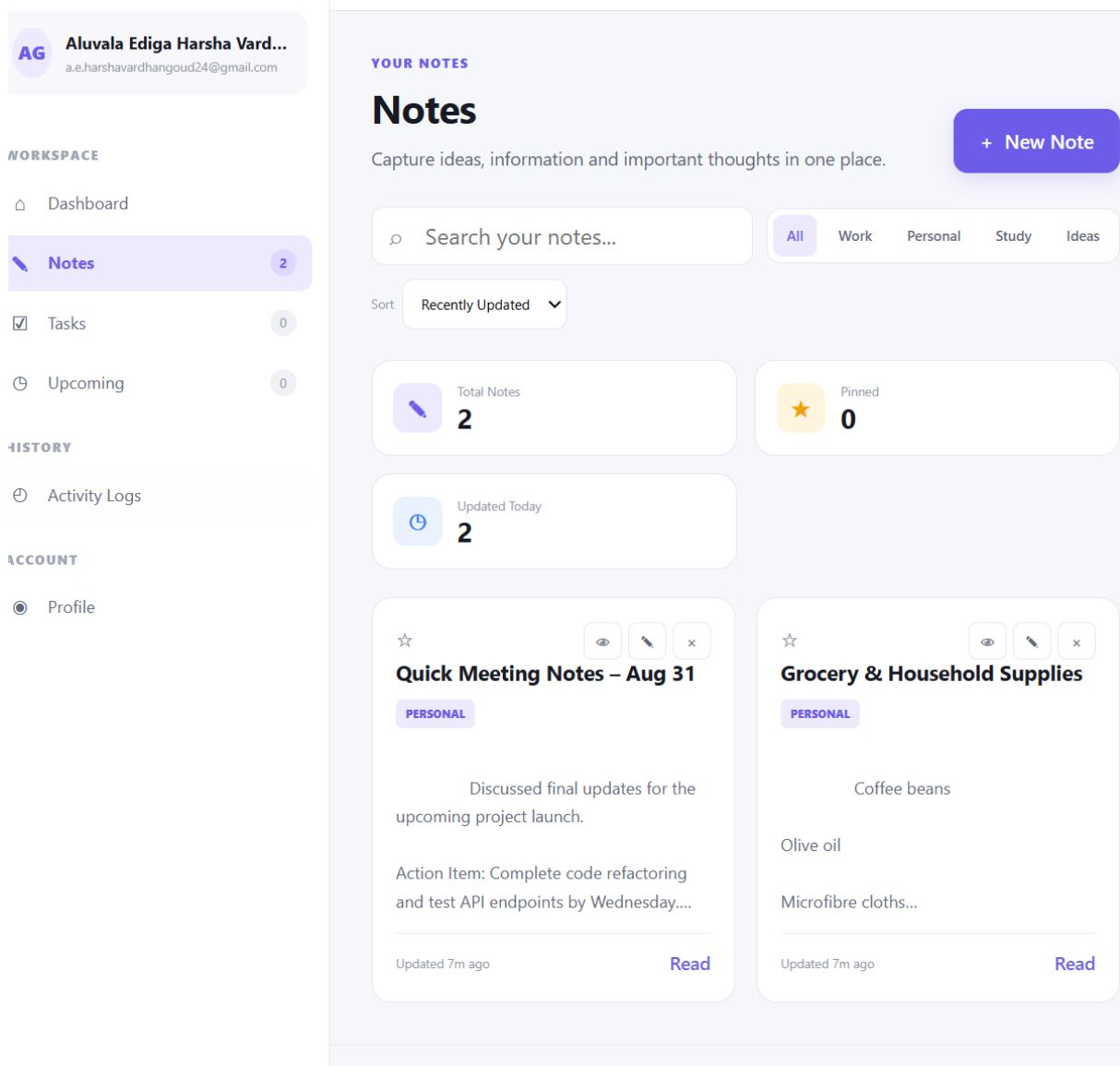


Figure A2. Notes page with cards, filters and statistics

Appendix A — Project Screenshots

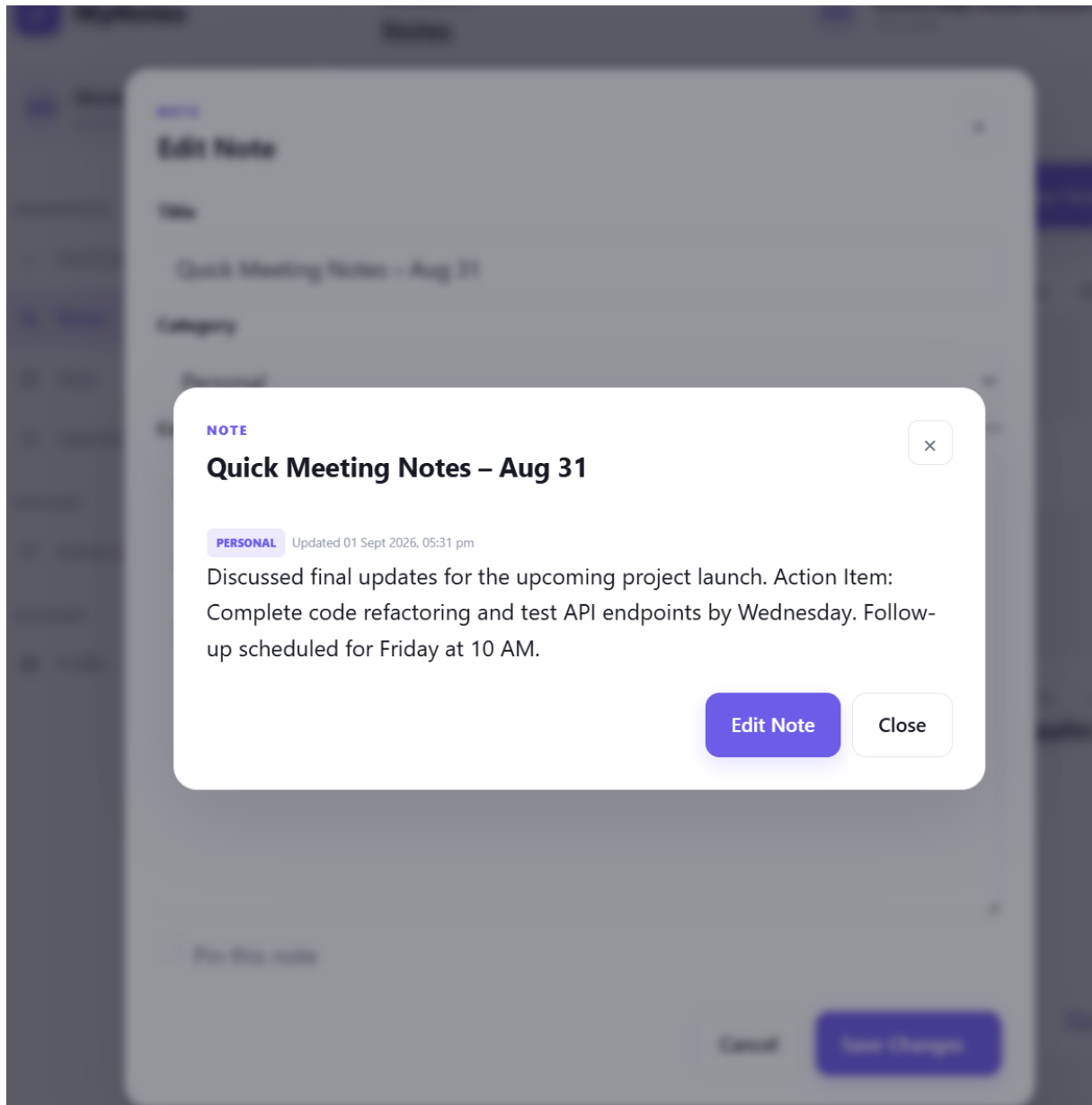


Figure A3. Note View modal

Appendix A — Project Screenshots

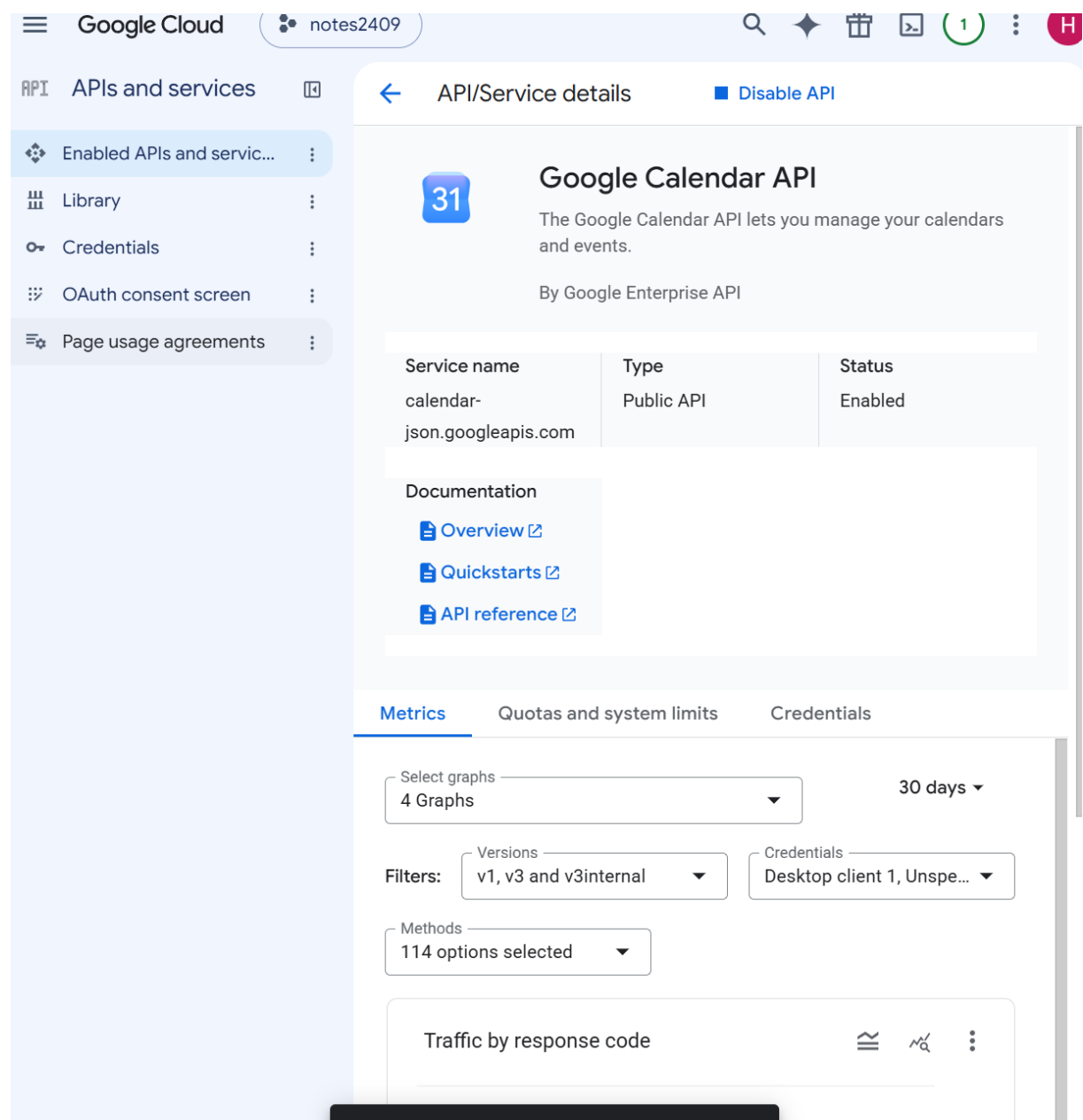


Figure A4. Google Calendar API project configuration

Appendix A — Project Screenshots

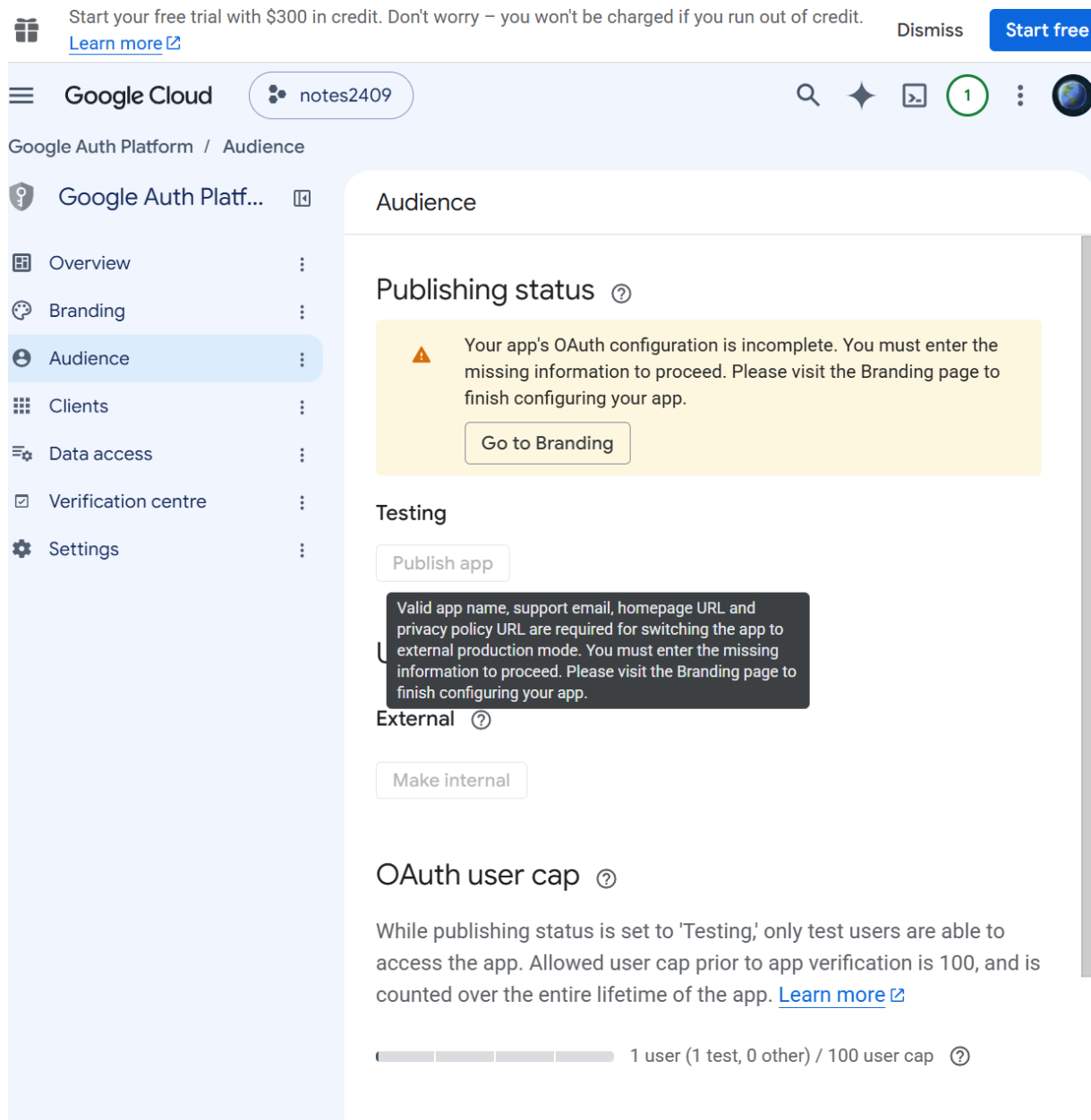


Figure A5. Google OAuth publishing / testing configuration

Appendix A — Project Screenshots

Google Auth Platform / Branding

Google Auth Platf...

Overview

Branding

Audience

Clients

Data access

Verification centre

Settings

Branding

Logo file to upload [Browse](#)

Upload an image, not larger than 1 MB on the consent screen that will help users recognise your app. Allowed image formats are JPG, PNG and BMP. Logos should be square and 120px by 120px for the best results.

App domain

To protect you and your users, Google only allows apps using OAuth to use Authorised Domains. The following information will be shown to your users on the consent screen.

Application home page *

Provide users a link to your home page

Application privacy policy link *

Provide users a link to your public privacy policy

Application Terms of Service link

Provide users a link to your public Terms of Service

Authorised domains ?

When a domain is used on the consent screen or in an OAuth client's configuration, it must be pre-registered here. If your app needs to go through verification, please go to the [Google Search Console](#) to check if your domains are authorised. [Learn more](#) about the authorised domain limit.

[+ Add domain](#)

Developer contact information

Email addresses *

sunriseseditsoffical249@gmail.com

These email addresses are for Google to notify you about any changes to your project.

[Save](#) [Discard changes](#)

Figure A6. Google OAuth application configuration

Appendix A — Project Screenshots

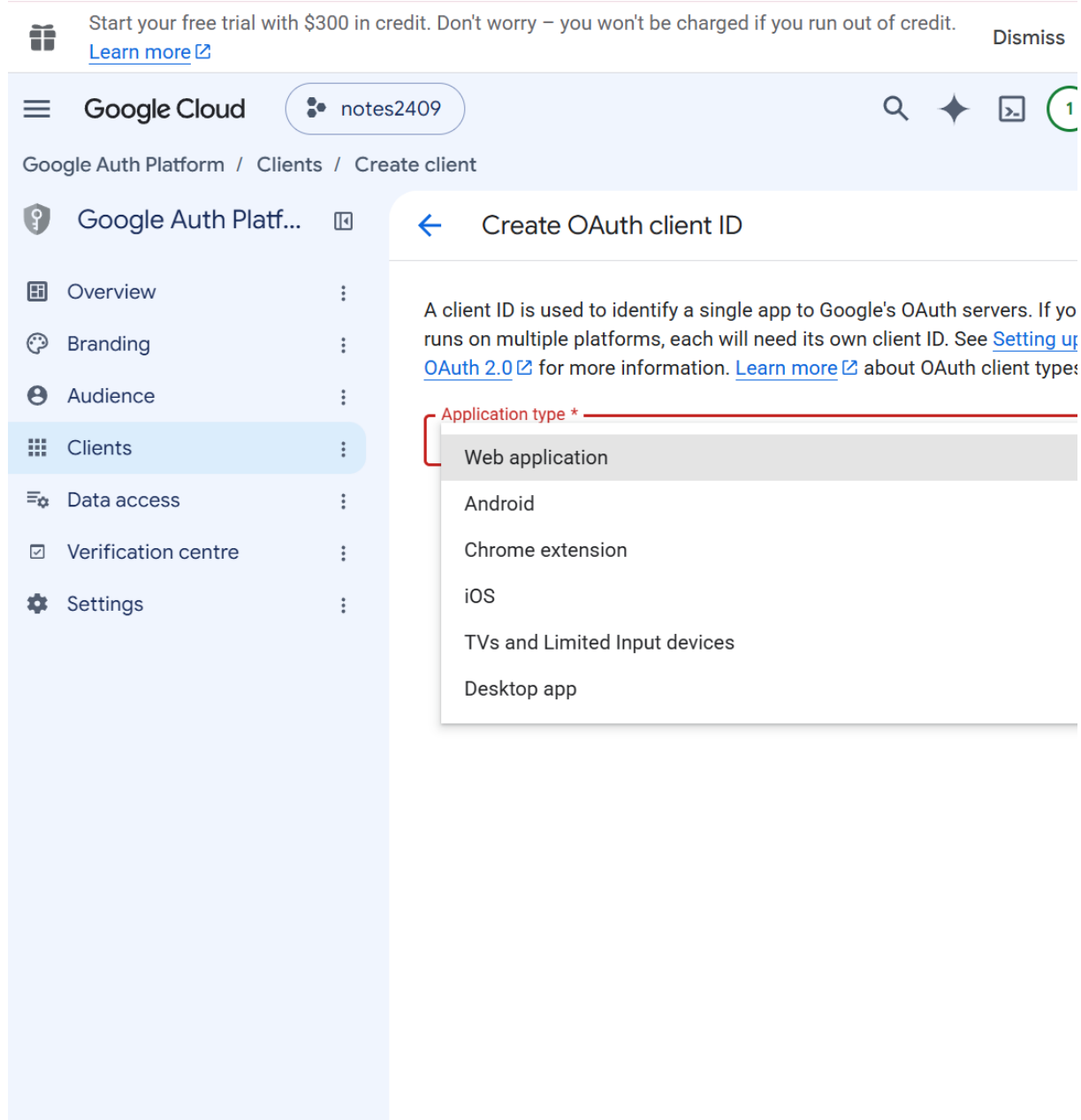


Figure A7. Google OAuth client / consent configuration

Appendix A — Project Screenshots

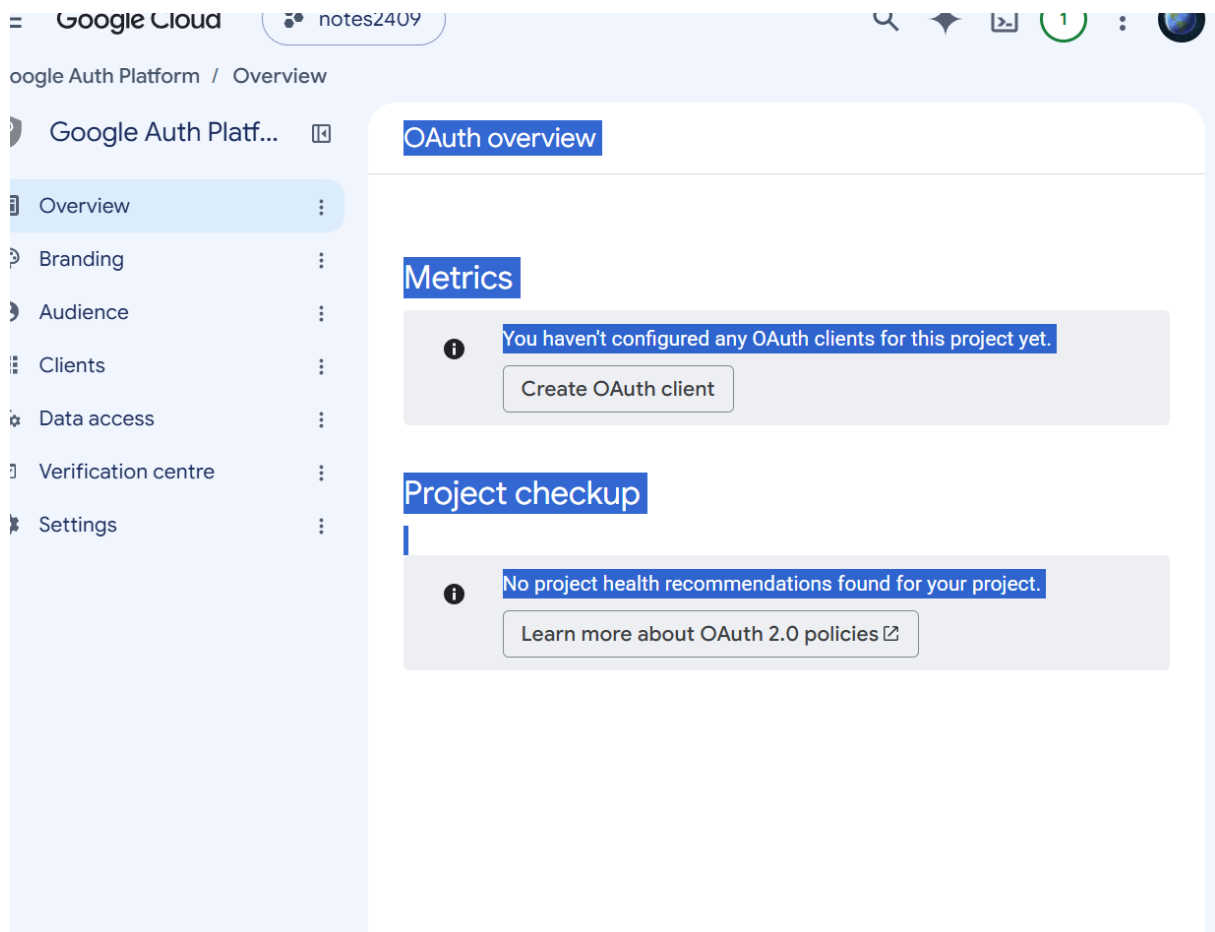


Figure A8. Google Cloud project checks / metrics

Appendix A — Project Screenshots

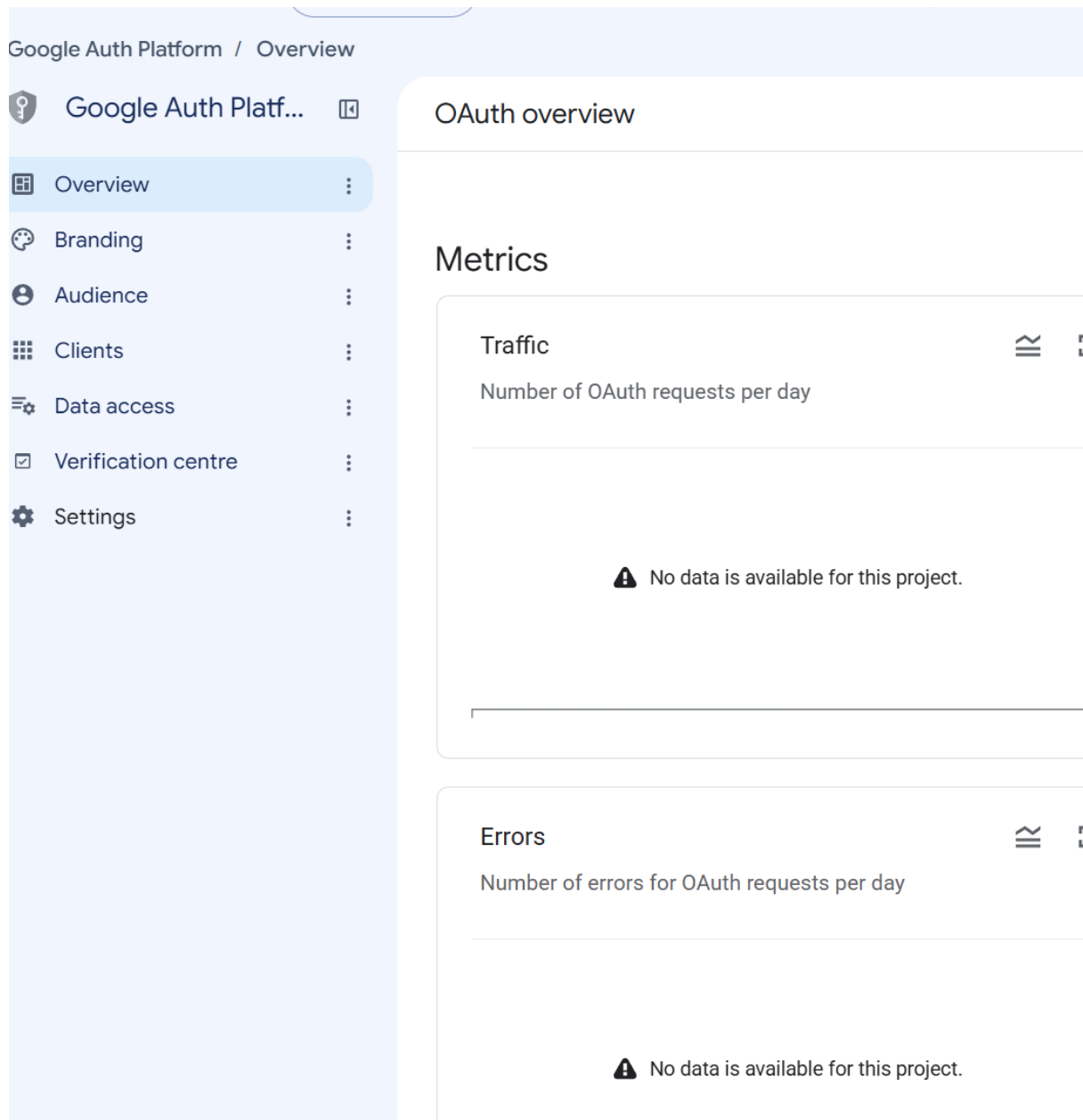


Figure A9. Google OAuth overview / metrics

Appendix A — Project Screenshots

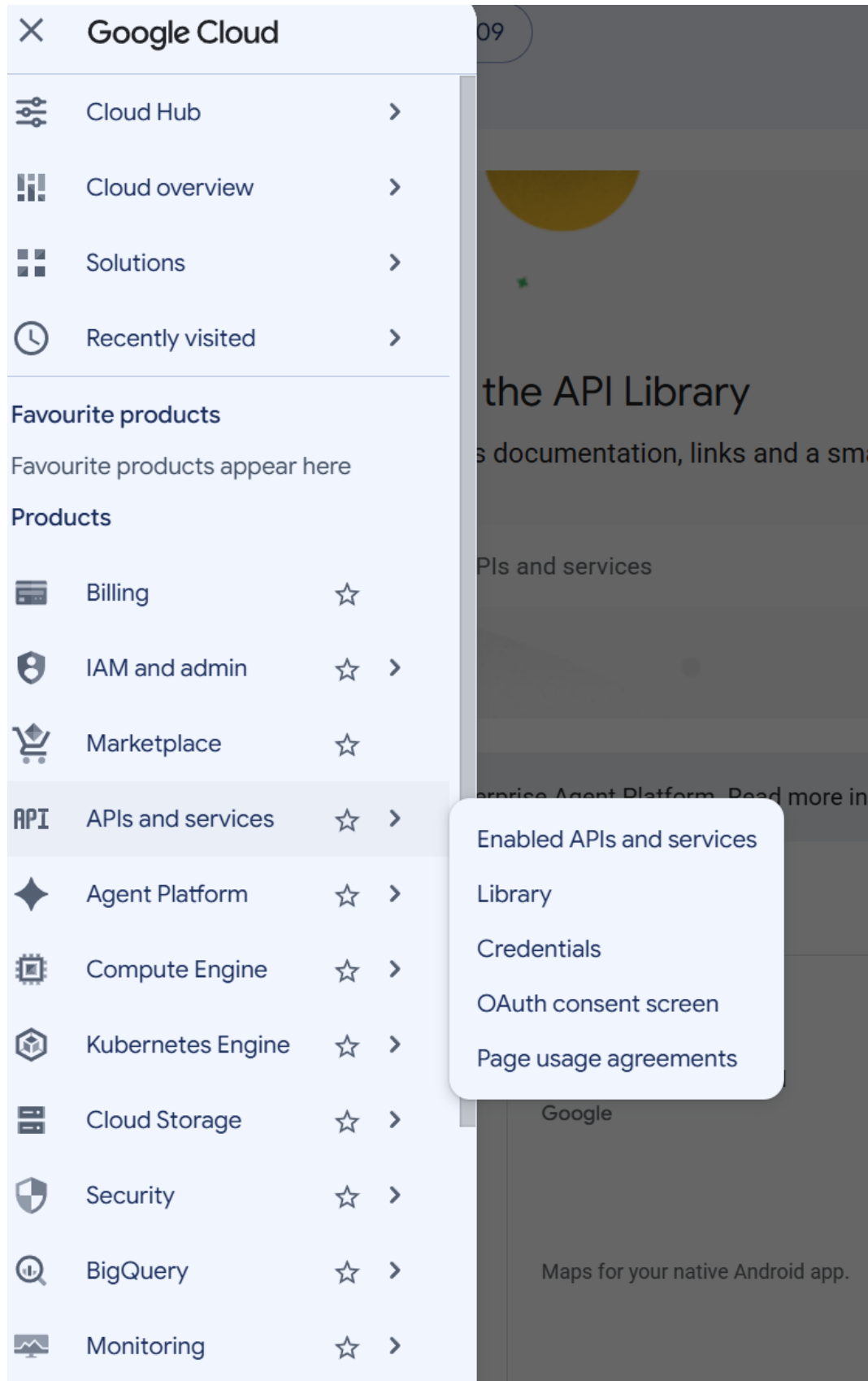


Figure A10. Google Cloud navigation / integration view

Appendix B — Architecture Reference Summary

Concern	Implemented / documented approach
Frontend	Jinja HTML + CSS + page-specific JavaScript
Backend	Flask route/controller layer
Persistence	MongoDB Atlas
Authentication	Session-based authenticated routes
Email	SMTP / Flask-Mail workflows
Calendar	Google OAuth + Calendar API
Calendar reminders	Event-specific popup overrides at 10 and 5 minutes
Deployment	Render + Gunicorn
Future AI	MCP roadmap only

Google Calendar documentation confirms that event reminders are expressed as minutes before event start and that event-specific overrides can replace calendar defaults. The project's 10-minute and 5-minute popup configuration follows that model.

Appendix C — Documentation References

Project-derived material was based on the MyNotes & Tasks source, HTML/JavaScript shared during development, project screenshots, README material and the existing project documentation artifact.

External technical reference:

Google Calendar API — Reminders and notifications:

<https://developers.google.com/workspace/calendar/api/concepts/reminders>

Google Calendar API — Events resource:

<https://developers.google.com/workspace/calendar/api/v3/reference/events>

Google Calendar API — Create events:

<https://developers.google.com/workspace/calendar/api/guides/create-events>

These external references were used only to describe the documented Calendar reminder/event behavior accurately.